

Redes Neuronales y Aprendizaje Profundo / Taller #1

Curso de Deep Learning

Universidad EAFIT

1. Parte A – Preguntas conceptuales

1.1 ¿Cuál es la diferencia funcional entre el Perceptrón y la Regresión Lineal, si ambos comparten la misma estructura $z = w \cdot x + b$?

1.2 En el modelo del perceptrón, ¿por qué es necesario el término de bias (b)? Explica qué pasaría geométricamente con la frontera de decisión si siempre se forzara $b = 0$.

1.3 En el algoritmo del perceptrón, la regla de actualización es:

$$\text{si } y^{[i]} \cdot (w \cdot x^{[i]}) \leq 0 \rightarrow w = w + y^{[i]} \cdot x^{[i]}$$

$$\text{si } y^{[i]} \cdot (w \cdot x^{[i]}) > 0 \rightarrow \text{no hacer nada}$$

¿Qué representa la condición $y^{[i]} \cdot (w \cdot x^{[i]}) \leq 0$? ¿Por qué esa condición indica una mala clasificación?

1.4 Explicar la diferencia entre las tres variantes del algoritmo de aprendizaje vistas en clase:

- Modo **on-line** (SGD puro, actualiza con cada dato)
- Modo **batch** (acumula el gradiente de todo el dataset y actualiza una sola vez por época)
- Modo **minibatch** (actualiza usando subconjuntos pequeños del dataset)

1.5 El "Fun Fact" de Minsky sobre el perceptrón que distinguía tanques camuflados en realidad estaba midiendo la cantidad de luz de la imagen. ¿Qué concepto de Machine Learning ilustra esta anécdota? ¿Cómo se relaciona con la diferencia entre *accuracy* y *aprender la relación correcta*?

1.6 En el algoritmo del Perceptrón, la actualización de pesos se decide únicamente con la condición $y^{[i]} \cdot (w \cdot x^{[i]}) \leq 0$, sin calcular explícitamente una magnitud de error. En cambio,

algoritmos como el gradiente descendente (por ejemplo, en ADALINE) sí actualizan los pesos en función de un error explícito $e^{[i]} = y^{[i]} - \hat{y}^{[i]}$

- ¿Por qué el Perceptrón puede decidir si actualizar o no sin necesitar una cantidad de error numérica, sino solo un criterio de "bien/mal clasificado"?
- ¿Qué ventaja aporta introducir el error como una magnitud continua (como hace el gradiente descendente) en comparación con la regla binaria del Perceptrón?

1.7 Supón que estás entrenando un modelo con gradiente descendente, y en cierta iteración el gradiente de la función de pérdida respecto a los pesos es $\nabla L(w) = [4, -2]$. La regla de actualización es $w := w - \eta \cdot \nabla L(w)$, con $\eta = 0.1$ y w actual = $[1, 1]$.

- ¿el primer peso (w_1) va a aumentar o disminuir? ¿y el segundo (w_2)?
- ¿Por qué el algoritmo se mueve en la dirección opuesta al gradiente y no en la misma dirección que él?

2. Parte B – Problemas prácticos:

2.1 Dado un perceptrón con pesos $w = [0.5, -0.3]$, bias $b = 0.1$, y función de activación escalón (1 si $z \geq 0$, 0 si $z < 0$). Calcula la salida $\hat{y}$ para las siguientes entradas:

a. $x = [1, 1]$

b. $x = [2, 3]$

c. $x = [-1, 0]$

2.2 Tienes un perceptrón con $w = [0, 0]$, $b = 0$. Aplica la regla de aprendizaje del perceptrón (modo on-line) sobre el siguiente dataset, en el orden dado para 1 época.

x_1	x_2	etiqueta
1	1	+1
-1	1	-1
0	-1	-1

Muestra el valor de w y b después de procesar cada ejemplo (recuerda incluir el bias como parte de x con un 1 fijo, o actualízalo aparte según la versión del algoritmo que uses).

2.3 Considere el siguiente conjunto de datos:

x y

1 2

2 3

3 5

4 6

Usando las fórmulas de mínimos cuadrados vistas en clase. Calcular manualmente β y β_0 . Escribe la ecuación final de la recta $\hat{y} = \beta_0 + \beta x$.

$$\beta = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}; \quad \beta_0 = \bar{Y} - \beta \bar{X}$$

2.4 Un paso de gradiente descendente para el mismo dataset del punto 2.3. Suponer que inicialmente $w = 0$, $b = 0$, $\eta = 0.01$, y usando solo el primer dato ($x=1$, $y=2$) en modo on-line.

- Calcular la predicción $\hat{y}$ inicial.
- Calcular el error ($y - \hat{y}$).
- Calcular el gradiente respecto a w y respecto a b usando $\partial L / \partial w = (\hat{y} - y) \cdot x$.
- Actualiza w y b con la regla $w := w + \eta \cdot (-\partial L / \partial w)$.

2.5 Considere un modelo de regresión lineal entrenando mediante **Batch Gradient Descent** (ver figura 1), dado por $\hat{y} = wx + b$. Donde, la función de costo a minimizar es:

$$J(w, b) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2.$$

Los gradientes para w y b están dados por

$$\frac{\partial J}{\partial w} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_i$$

$$\frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

- Usando los datos presentados en la tabla, y asumiendo que inicialmente $w_0 = 0$, $b_0 = 0$ y $\eta = 0.1$, después de 1 época (1 paso de Batch GD), los nuevos parámetros (w_1 , b_1) son:

Datos	x_i	y_i
1	1	2
2	2	4
3	3	6

Resultado: w , b
 $w := 0 \in \mathbb{R}^m$, $b := 0$;

```

for t = 1,...,T do
     $\Delta w := 0$ ,  $\Delta b := 0$ ;

    for i = 1,...,n do
        calcular predicción  $\hat{y}^{(i)} = w^T x^{(i)} + b$ ;
        calcular error  $e^{(i)} = \hat{y}^{(i)} - y^{(i)}$ ;
        // ACUMULAR gradientes
         $\Delta w := \Delta w + e^{(i)} \cdot x^{(i)}$ ;
         $\Delta b := \Delta b + e^{(i)}$ ;
    end

    // Promediar gradientes
     $\Delta w := \Delta w / n$ ;
     $\Delta b := \Delta b / n$ ;
    // ACTUALIZAR parámetros UNA SOLA VEZ por época
     $w := w - \alpha \cdot \Delta w$ ;  $b := b - \alpha \cdot \Delta b$ ;
end

```

Figura 1. Seudocódigo Batch Gradient Descent.

- Usando (w_1, b_1) , evalúe la predicción para $x = 1$: $\hat{y}(1) = w_1 \cdot 1 + b_1$
 Comparada con la predicción inicial $\hat{y}_0(1) = 0$, la nueva predicción:
 - Mejora (queda más cerca de $y = 2$)
 - Empeora (queda más lejos de $y = 2$)
 - No cambia
 - No se puede saber

3. Parte C – Ejercicios de Código:

3.1 Implementar Regresión Lineal con Gradiente Descendente (modo batch): Implementar una clase LinearRegressionGD que entrene con todo el dataset en cada actualización

```
class LinearRegressionGD:
    def __init__(self, num_features, lr=0.01):
        self.w = np.zeros(num_features)
        self.b = 0.0
        self.lr = lr

    def predict(self, X):
        # TODO
        pass

    def train(self, X, y, epochs=100):
        # TODO: acumular gradiente sobre todo el dataset,
        # actualizar w y b una vez por época
        Pass
```

Generar un dataset sintético con `numpy.random` (por ejemplo $y = 3 \cdot x + 5 + \text{ruido}$), entrena tu modelo, y grafica con `matplotlib`: Los datos originales (`scatter`) y la recta aprendida. Evaluar una métrica que permita calcular el error.

3.2 Comparar Batch vs. Minibatch vs. Estocástico Usando el mismo dataset sintético del ejercicio 3.1, implementar las tres variantes de entrenamiento (batch, minibatch con $k = 8$, y on-line/estocástico). Graficar la evolución de la función de pérdida L por época para las tres variantes en un mismo gráfico. ¿cuál converge más rápido? ¿cuál es más "ruidosa"?

3.3 Usando la implementación del punto 3.1, entrenar el mismo modelo con tres tasas de aprendizaje distintas: $\eta = 0.001$, $\eta = 0.01$, $\eta = 0.5$. Graficar la curva de pérdida para cada una. ¿Qué le sucede al entrenamiento cuando η es muy alta?

3.4 Implemente el código que se entrega a continuación, tener en cuenta que se debe completar cada TODO:

```
import torch
import torch.nn as nn
import numpy as np

# Dataset XOR / XNOR
X_xor = torch.tensor([[0.,0.],[0.,1.],[1.,0.],[1.,1.]])
y_xor = torch.tensor([[0.],[1.],[1.],[0.]])
y_xnor = torch.tensor([[1.],[0.],[0.],[1.]])

class MLP(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        # TODO: define una capa oculta (nn.Linear) + activación no lineal
        # y una capa de salida (nn.Linear) + activación logística
        self.hidden = nn.Linear(2, hidden_size)
        self.activacion = nn.Sigmoid() # también puedes probar nn.Tanh()
        self.salida = nn.Linear(hidden_size, 1)
        self.sigmoid_salida = nn.Sigmoid()

    def forward(self, x):
        # TODO: implementa el forward pass encadenando las capas
        pass

# TODO: crea el modelo con un hidden_size que consideres razonable
modelo = MLP(hidden_size=?)

criterio = nn.BCELoss() # pérdida de entropía cruzada binaria
optimizador = torch.optim.SGD(modelo.parameters(), lr=?) # TODO: ajusta lr

epochs = ? # TODO: ajusta el número de épocas

for epoch in range(epochs):
    optimizador.zero_grad()
```

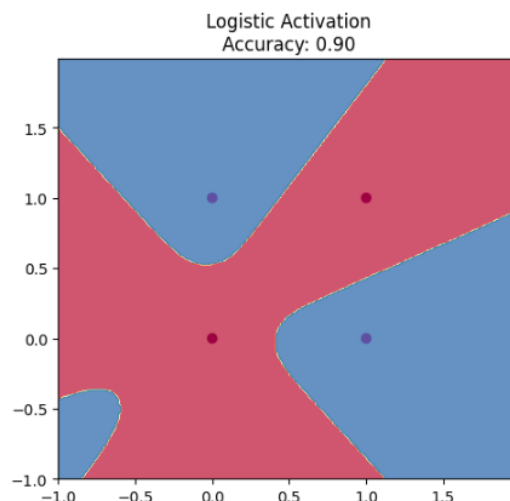
```
salida = modelo(X_xor)
perdida = criterio(salida, y_xor)
perdida.backward()
optimizador.step()

if epoch % 500 == 0:
    print(f"Epoch {epoch} — Loss: {perdida.item():.4f}")
```

Evaluación

```
with torch.no_grad():
    predicciones = (modelo(X_xor) > 0.5).float()
    accuracy = (predicciones == y_xor).float().mean()
    print("Accuracy:", accuracy.item())
```

- Ajustar `hidden_size`, `lr` y `epochs` hasta obtener `accuracy = 1.0` para XOR y para XNOR.
- Documentar en una tabla al menos 3 combinaciones de hiperparámetros que y el `accuracy/loss` final obtenido.
- ¿Qué le pasa a la pérdida (`perdida.item()`) a lo largo del entrenamiento si el `hidden_size` es demasiado pequeño (por ejemplo, 1 neurona oculta)?
- Construir una función que grafique la región de decisión aprendida, similar a la Figura de referencia (frontera curva no lineal con los 4 puntos superpuestos).



3.5 Ingresar a <https://www.sfu.ca/~ssurjano/optimization.html> y selecciona una función de prueba de 2 variables (por ejemplo: *Sphere*, *Booth*, *Matyas*, *Beale*, *Himmelblau*, o si quieres un reto mayor, *Rosenbrock* o *Rastrigin*).

- a. Escribe la fórmula matemática de la función seleccionada, $f(x,y)$.
- b. Identificar en la página el(los) mínimo(s) global(es) conocido(s) y su valor $f(x^*, y^*)$ — esto es necesario para verificar resultados.
- c. Calcular a mano las derivadas parciales $\partial f/\partial x$ y $\partial f/\partial y$ (el gradiente $\nabla f(x,y)$).
- d. Implementar el algoritmo del descenso del gradiente desde cero (sin librerías de optimización).
- e. Ajustar el punto inicial (semilla) y la tasa de aprendizaje (lr) hasta lograr que el algoritmo converja al mínimo global conocido en menos de 50 iteraciones, con un error $|z_{\text{encontrado}} - z_{\text{mínimo_conocido}}| < 0.01$.
- f. Graficar la superficie (o curvas de nivel) de $f(x,y)$ junto con la trayectoria que siguió el algoritmo. Para ello, utilizar al menos 3 puntos iniciales distintos (algunos cerca y otros lejos del mínimo). ¿En todos los casos convergió al mínimo global? Si no, explicar por qué?, ¿tiene mínimos locales?). Además, probar con al menos 2 tasas de aprendizaje distintas para el mismo punto inicial. ¿Qué ocurrió con una lr demasiado alta? ¿Y con una demasiado baja? Relaciona esto con lo discutido en la pregunta 1.10 del taller.